

Track 12 — Mailversand, Rückkehr-URLs und der fehlende Miet-Storno

Datum: 2026-08-28 **Ausgangsstand:** 92ddef4 (Track 11), 1083 Tests **Endstand:** 1168 Tests (85 neu), Typecheck grün, ESLint ohne Befund

Vorgehen

Systematischer Abgleich der im Auftrag benannten Bereiche gegen die tatsächlichen Quellen: Stripe-Checkout-Validierung, Webhook-Signaturprüfung, Admin-Autorisierung, E-Mail-Vorlagen, Mietstrecke End-to-End, `payment_status`-Konsistenz, Rate-Limiting.

Zwei Bereiche waren nach Prüfung in Ordnung und werden hier nur der Vollständigkeit halber genannt:

- **Webhook-Signatur.** `stripe.webhooks.constructEvent` mit Rohtext, fehlender Header und fehlendes Secret werden getrennt abgewiesen. Kein Befund.
- **Admin-Autorisierung.** Alle 16 Routen unter `/api/admin/**` prüfen die Rolle selbst, zusätzlich greift die Middleware seit Track 11 auch auf dem `/api/admin`-Präfix. `/api/setup/promote-admin` steht zwar in der Public-Whitelist, ist aber durch Setup-Key, Mindestlänge, zeitkonstanten Vergleich und Rate-Limit gedeckt. Kein Befund.
- **CSRF.** Das NextAuth-Session-Cookie ist `sameSite: 'lax'`; fremde Seiten können keine zustandsändernden POSTs auslösen. Kein Befund.

Ein vermuteter P0 hat sich in der Live-Sonde **nicht** bestätigt und wird unten unter Befund 7 als das behandelt, was er wirklich ist.

Befunde

1 — P0: `/api/email` war ein Phishing-Versand mit unserer Absenderreputation

Drei Dinge trafen zusammen, jedes für sich unauffällig:

a) Die Vorlagen escapeten drei Felder nicht. In `sendBookingConfirmation` und `sendBookingReminder` standen `bookingId`, `startTime` und `endTime` roh im HTML — während `salonName`, `serviceName`, `customerName` und `staffName` direkt daneben durch `esc()` liefen. Es war kein Konzeptfehler, sondern drei vergessene Felder. Dazu kamen der `<title>` im Grundlayout, die URL in `goldButton()`, der `formatDate`-Rückfall, der Dokumenttyp im Compliance-Hinweis und die Reset-URL im Klartexthinweis.

b) Die Route nahm Inhalt UND Empfänger aus dem Request. `POST /api/email` validiert die Form, aber nicht die Herkunft: `to` ist frei wählbar, sämtliche Inhaltsfelder kommen aus dem Body. Ein Bezug zwischen Absender und Empfänger wird nirgends verlangt.

c) Die Rolle `anbieter` durfte `booking_confirmation` senden. Diese Rolle ist öffentlich selbst zu beschaffen — `POST /api/register-provider` steht in `publicPrefixes` und vergibt sie ohne Prüfung.

Zusammen: Wer sich als Anbieter registriert, konnte beliebiges Markup samt Link in eine Mail schreiben, die von `noreply@chairmatch.de` kommt, DKIM-signiert ist, unser Layout trägt — und an jede beliebige Adresse geht. Das ist die überzeugendste Form von Phishing, die eine Domain hergibt, und sie hätte die Zustellbarkeit der Domain mitgenommen.

Behoben. `esc()` deckt jetzt auch `'` ab (Ausbruch aus einfach-gequoteten Attributen); `safeUrl()` lässt in `href` nur noch `http/https`; alle Interpolationen in den Vorlagen sind escapet. Die Rollen-Allowlist steht auf `admin / super_admin`, mit einer Notiz, wie ein Anbieter-Nachversand richtig aussähe: eine eigene Route, die die Buchung über ihre ID **nachschlägt** und den Empfänger aus der Buchung nimmt. Dazu ein eigenes Rate-Limit (10/min statt der generischen 60/min der Middleware).

Die Route hat übrigens keinen einzigen Aufrufer in der Anwendung.

2 — P1: Buchungs-Mails verlinkten eine Seite, die es nicht gibt

Bestätigung und Erinnerung trugen beide den Button „Buchung ansehen“ auf

```
https://www.chairmatch.de/booking/${details.bookingId}
```

Die Route unter diesem Pfad ist `src/app/(protected)/booking/[salonId]/page.tsx` — das **Buchungsformular eines Salons**. Mit einer Buchungs-ID darin lädt die Seite einen Salon, den es nicht gibt. Der einzige Knopf in der Bestätigungsmail führte also ins Leere.

Behoben. Beide Mails verlinken `/termine` — die Seite, die seit Track 6 die echten Buchungen des angemeldeten Kunden lädt und über `POST /api/bookings/[id]/cancel` auch absagt.

3 — P1: Der `origin` -Header steuerte alle Stripe-Rückkehr-URLs

An sechs Stellen stand:

```
const origin = req.headers.get('origin') || 'https://www.chairmatch.de'
```

und der Wert ging ungeprüft als `success_url / cancel_url` in die Checkout-Session bzw. als `return_url / refresh_url` in das Connect-Onboarding. `Origin` setzt bei einem Browser der Browser — diese Endpunkte sind aber nicht auf Browser angewiesen; ein `curl -H 'Origin: ...'` genügte.

Das Ergebnis ist eine **echte**, von uns erzeugte Stripe-Session, gehostet auf `checkout.stripe.com`, mit unserem Produktnamen und unserem Betrag, die nach der Zahlung auf eine fremde Domain weiterleitet. Beim Connect-Onboarding wiegt es schwerer: dort landet der Anbieter nach Eingabe seiner Bank- und Ausweisdaten auf der Seite des Angreifers.

Behoben. `src/lib/app-origin.ts`: Der Header darf nur noch **bestätigen**, welcher unserer eigenen Ursprünge genommen wird, und keinen neuen mehr einführen. Erlaubt sind `chairmatch.de`, `www.chairmatch.de`, `NEXT_PUBLIC_APP_URL`, das eigene Vercel-Deployment (`VERCEL_URL / VERCEL_BRANCH_URL` — aus der Umgebung, nicht aus dem Request) und `localhost`

ausschließlich in der Entwicklung. Ein fremder Wert führt nicht zu einem Fehler, sondern schlicht zurück zu uns.

4 — P1: Das Einwilligungs-Protokoll speicherte die IP lesbar

Zwei Stellen, ein Muster.

`POST /api/auth/register` schrieb in die Spalte `ip_hash`:

```
const ipHash = ip ? Buffer.from(ip).toString('base64').slice(0, 32) : null
```

Base64 ist eine Kodierung, kein Hash. `MTk4LjUxLjEwMC4yMw==` ist in einer Zeile zurückzurechnen. In `consent_logs` lag damit die IP jeder registrierten Person im Klartext, nur anders geschrieben, unter einem Spaltennamen, der das Gegenteil behauptet.

`POST /api/register-provider` schrieb die IP gleich unverändert nach `audit_logs.details.ip`.

Gedeckt war das von zwei grünen Tests, und beide haben den Befund nicht übersehen, sondern festgeschrieben:

```
// auth-flow.test.ts – prüft die Schreibweise, nicht die Umkehrbarkeit
expect(String(consent.ip_hash)).not.toContain('198.51.100.23')

// provider-onboarding.test.ts – verlangte den Klartext ausdrücklich
expect(consent?.details).toMatchObject({ ..., ip: '198.51.100.7' })
```

Behoben. `src/lib/ip-hash.ts`: HMAC-SHA-256 mit serverseitigem Geheimnis (`CONSENT_IP_SALT`, Rückfall auf das Auth-Geheimnis). Ein bloßer SHA-256 ohne Schlüssel wäre hier wertlos — der gesamte IPv4-Raum ist in Minuten durchgerechnet. Ohne Geheimnis bleibt die Spalte leer statt schwach gefüllt. Beide Bestandstests sind umgeschrieben und prüfen jetzt die Eigenschaft statt der Schreibweise: Hex-Form, Determinismus, Trennschärfe und dass weder base64 noch base64url noch hex noch URL-Dekodierung zurückführen.

Das Rate-Limit in `/api/register-provider` braucht die Adresse weiterhin und behält sie — sie verlässt nur den Aufrufer nicht mehr Richtung Protokoll.

5 — P2: Die Erinnerungsmail versprach pauschal 24 Stunden Gratisstorno

Musst du umbuchen? Du kannst den Termin bis 24h vorher kostenlos stornieren.

An **jeden** Empfänger, während die Frist pro Salon in `booking_policies.cancellation_hours` gepflegt wird und `cancelBooking` sie auch tatsächlich ausliest. Track 6 hat genau diesen Satz aus dem Buchungsformular entfernt; in der Mail stand er weiter.

Behoben. `BookingEmailDetails` trägt jetzt ein optionales `cancellationHours`. Ohne Wert nennt die Mail keine Zahl, sondern verweist auf die Terminliste, wo die Frist am Termin steht. Mit Wert nennt sie den echten.

Nebenbefund, bewusst nicht angefasst: Diese Mail wird derzeit überhaupt nicht verschickt.

`sendBookingReminder` hat keinen Aufrufer außer `/api/email`, und in `vercel.json` steht kein Erinnerungs-Cron. Eine Erinnerungsstrecke zu bauen hieße, echte Mails an echte Kunden auszulösen — das gehört nicht in einen Härte-Track.

6 — P2: Die bezahlte Miete war eine Einbahnstraße

Der Payout-Cron begründet die Escrow-Zurückhaltung so:

„Bezahlte Miet-Transaktionen werden NICHT sofort an den Anbieter transferiert, sondern erst wenn der Mietbeginn erreicht ist — das schützt Mieter bei No-Show/Storno vor Mietantritt.“

Das Zurückhalten gab es wirklich. Den Storno, für den es da ist, nicht: unter `/api/rental-bookings` lagen ausschließlich `POST` (anlegen) und `GET` (auflisten), einen `[id]`-Handler gab es überhaupt nicht. Weder Mieter noch Vermieter kamen aus einer bezahlten Buchung heraus. Am Starttag zahlte der Cron dann aus.

Behoben. `POST /api/rental-bookings/[id]/cancel`, angehängt an den Mieter-Verlauf unter `/mieter/mein-bereich/verlauf`.

Was die Route bewusst **nicht** tut:

- **Keine erfundene Stornogebühr.** `rental_bookings` hat live keine Spalte, die eine aufnehmen könnte (Sonde 2026-08-28: `cancelled_at`, `cancellation_reason` und `refund_cents` existieren alle nicht), und einen Satz gäbe es nirgends nachzulesen. Erstattet wird voll oder gar nicht.
- **Kein Storno nach Mietbeginn.** Was in einem laufenden Mietverhältnis anteilig zurückzugeben wäre, ist eine kaufmännische Entscheidung mit einer Zahl darin. Ab dem Starttag verweist die Antwort auf den Support.
- **Keine Erstattung dessen, was der Anbieter schon hat.** Steht an der Plattform-Transaktion eine `stripe_transfer_id`, ist das Geld auf dem Connect-Konto; ein Refund liefe dann aus unserer Tasche. Der Fall ist vor Mietbeginn nicht zu erwarten, wird aber geprüft statt vorausgesetzt.
- **Kein Storno, wenn die Erstattung scheitert.** Eine stornierte Buchung ohne Geld zurück ist der schlechteste aller Zustände.

Der Storno-Grund landet im Audit-Log, nicht an der Buchung — eine Spalte dafür gibt es live nicht. Der Knopf in der Oberfläche erscheint unter genau denselben Bedingungen, unter denen die Route ausführt; ein Knopf, der zuverlässig 409 erntet, wäre dieselbe Sorte Versprechen wie der Kommentar im Cron.

7 — P3: Das Testharness führte `rental_bookings` unvollständig

`src/test/live-schema.ts` sagt für diese Tabelle ausdrücklich „der volle Ist-Zustand“ zu, listete aber elf Spalten ohne `stripe_payment_intent` — obwohl der Webhook sie seit Track 6 liest und schreibt (Doppelzahlungs-Guard, `charge.refunded`).

Das sah zunächst nach einem P0 aus: ein `select` auf eine nicht existierende Spalte läuft in PostgREST in 42703, und dann hätte **jede** Miet-Zahlung still gescheitert. Die Live-Sonde vom 2026-08-28 sagt etwas anderes — die Spalte existiert:

```
OK      rental_bookings.stripe_payment_intent
FEHLT   rental_bookings.cancelled_at
FEHLT   rental_bookings.cancellation_reason
FEHLT   rental_bookings.refund_cents
```

Der Produktivcode war also richtig, die Liste falsch. Ein Test, der diese Liste durchsetzt, hätte korrekten Code als Fehler gemeldet.

Behoben. Spalte nachgetragen in `live-schema.ts` und in `scripts/schema-probe.sh`, Sondendatum auf 2026-08-28.

8 — P3: Der geteilte Test-Fake kannte `.not()` nicht

`src/test/fake-supabase.ts` hatte keine `.not()`-Methode. Der Payout-Cron wählt seine Auszahlungskandidaten aber genau damit aus:

```
.not('provider_user_id', 'is', null)
.not('stripe_payment_intent_id', 'is', null)
```

Wer dafür einen Test schreiben wollte, bekam `...eq(...).not is not a function`. Die Auszahlungsauswahl war nicht prüfbar — nicht weil sie schwer wäre, sondern weil das Werkzeug fehlte. (Eine `.not()` gab es im Repo, aber in der **zweiten** Fake-Implementierung, der inline in den e2e-Tests gebauten.)

Behoben. `.not(spalte, operator, wert)` nachgetragen, mit echter Auswertung: der Zellvergleich ist in `compareCell()` ausgelagert und wird negiert. Eine Attrappe, die jeden Filter durchwinkt, wäre schlimmer als keine Methode — der Test wäre dann grün und falsch.

Tests

1083 → 1168 (85 neu), alle grün.

Datei	Tests	Deckt ab
<code>src/lib/__tests__/email-escaping.test.ts</code>	20	Befunde 1, 2, 5 — rendert die echten Vorlagen über den echten Versandweg und fängt das HTML am Resend-Client ab
<code>src/lib/__tests__/app-origin.test.ts</code>	20	Befund 3 — Lookalike-Domains, Subdomain- und Prefix-Tricks, Vercel-Preview, localhost nur in der Entwicklung
<code>src/lib/__tests__/ip-hash.test.ts</code>	13	Befund 4, inkl. eines Tests, der belegt, warum der alte Test den Defekt nicht sehen konnte
<code>src/app/api/rental-bookings/__tests__/cancel.e2e.test.ts</code>	25	Befund 6 — Berechtigung, Frist, Erstattung, Protokoll; läuft gegen das Produktionsschema
<code>src/__tests__/mail-relay-und-rueckkehr-urls.test.ts</code>	7	Statische Wächter gegen die Rückkehr aller vier Muster

Zwei Bestandstests umgeschrieben, weil sie den Befund als Sollverhalten festgeschrieben hatten (`auth-flow.test.ts` , `provider-onboarding.test.ts`).

Offen, bewusst nicht angefasst

- **Es gibt keine Terminerinnerung.** Kein Cron, kein Aufrufer (Befund 5). Die Vorlage ist jetzt ehrlich, aber sie liegt brach.
- **POST /api/email hat keinen Aufrufer.** Der Endpunkt ist jetzt admin-only und escapet sauber; ob er bleiben soll, ist eine Produktentscheidung.
- **Storno einer laufenden Miete.** Braucht eine Regel für die anteilige Rückgabe — also eine Zahl, die jemand festlegen muss.
- **/api/cookies/consent** nimmt eine frei wählbare `sessionId` ohne Auth entgegen und setzt `ip_hash` gar nicht. Kein Schaden, aber das Protokoll belegt wenig.